

United International University
Department of Computer Science and Engineering
Data Structures and Algorithms Lab
Assignment — Linked List

Total Marks: 100 | **Problems:** 10 | **Language:** C / C++ / Java / Python
Student Name: _____ **ID:** _____ **Section:** _____

Instructions

- Implement each problem as a separate C/C++/JAVA/PYTHON file.
- Your linked list node must contain at least a data field and a next pointer.
- Do not use built-in List, LinkedList, or similar standard library collections for the core logic.
- Input format and output format must exactly match what is shown in each problem.
- Clearly write the problem number and your student ID as comments at the top of each file.
- Submissions without proper input/output matching will not receive full credit.
- **DO NOT copy from your friends, plagiarisms will be checked and will be hardly penalized!**
- **DO NOT use GPT or any other LLM to solve the problems. If you are detected using GPT, you will be marked in negative!**

Standard Node Structure

Use the following node structure across all problems (adapt syntax to your language):

```
// C example
struct Node {
    int data;
    struct Node* next;
};

// Java example
class Node {
    int data;
    Node next;
    Node(int d) { data = d; next = null; }
}
```

Problems

Problem 1: Build and Print a Train Route [10 marks]

● Easy

📌 **Real-World Scenario:** A railway scheduling system stores station names in the order trains visit them. Build the linked list of stations and print the route.

Task:

Read N station names and insert them in order to form a singly linked list. Then print all stations separated by " → ", ending with "END".

Input Format:

```
Line 1: N (number of stations,  $1 \leq N \leq 20$ )
Next N lines: one station name each (no spaces, max 20 chars)
```

Output Format:

```
Station1 → Station2 → ... → StationN → END
```

Sample Input / Output:

Input	Expected Output
4 Dhaka Comilla Chittagong CoxsBazar	Dhaka → Comilla → Chittagong → CoxsBazar → END
1 Sylhet	Sylhet → END
3 Rajshahi Pabna Sirajganj	Rajshahi → Pabna → Sirajganj → END

Problem 2: Count and Sum of Elements [10 marks]

● Easy

📌 **Real-World Scenario:** A bank maintains a linked list of daily transaction amounts. Find the total number of transactions and their sum.

Task:

Given a linked list of integers, print the count of nodes and the sum of all values.

Input Format:

```
Line 1: N (number of elements)
Line 2: N space-separated integers
```

Output Format:

```
Count: X
Sum: Y
```

Sample Input / Output:

Input	Expected Output
5 200 150 300 50 400	Count: 5 Sum: 1100
3 1000 2000 3000	Count: 3 Sum: 6000
1 750	Count: 1 Sum: 750

Problem 3: Insert at Any Position [10 marks]

● Easy

📌 **Real-World Scenario:** A playlist manager allows users to insert a new song at any position. Implement position-based insertion.

Task:

Build a linked list from N elements. Then insert a new element at a given 1-based position. Print the list before and after insertion.

Input Format:

```
Line 1: N
Line 2: N space-separated integers (the initial list)
Line 3: value pos (insert 'value' at position 'pos')
```

Output Format:

```
Before: e1 e2 ... eN
After:  e1 ... eK value eK+1 ...
```

Sample Input / Output:

Input	Expected Output
4 10 20 30 40 99 2	Before: 10 20 30 40 After: 10 99 20 30 40
3 5 15 25 10 1	Before: 5 15 25 After: 10 5 15 25
3 1 2 3 9 4	Before: 1 2 3 After: 1 2 3 9

Problem 4: Delete by Value [10 marks]

● Easy

 **Real-World Scenario:** A contact list app lets users remove a contact by name/number. Delete the first occurrence of a given value from the linked list.

Task:

Build a linked list. Delete the first node whose data matches the given value. If the value is not found, print "Not found". Print the list before and after deletion.

Input Format:

```
Line 1: N
Line 2: N space-separated integers
Line 3: value to delete
```

Output Format:

```
Before: ...
After: ... OR Not found
```

Sample Input / Output:

Input	Expected Output
5 10 20 30 20 40 20	Before: 10 20 30 20 40 After: 10 30 20 40
4 1 2 3 4 5	Before: 1 2 3 4 Not found

3 7 8 9 7	Before: 7 8 9 After: 8 9
-----------------	-----------------------------

Problem 5: Search and Report [10 marks]

● Easy

📌 **Real-World Scenario:** A student record system must locate a student by roll number and report their position in the list.

Task:

Given a linked list and a search key, find and print all 1-based positions where the key appears. If not found, print "Not found".

Input Format:

```
Line 1: N
Line 2: N space-separated integers
Line 3: key to search
```

Output Format:

```
Found at position(s): p1 p2 ... OR Not found
```

Sample Input / Output:

Input	Expected Output
6 10 30 20 30 40 30 30	Found at position(s): 2 4 6
4 5 10 15 20 7	Not found
3 100 200 100 100	Found at position(s): 1 3

Problem 6: Reverse the Delivery Route [10 marks]

● Medium

 **Real-World Scenario:** A food delivery app stored pickup stops in one direction, but the driver must now return. Reverse the linked list *in-place* (no auxiliary array) to get the reverse route.

Task:

Reverse a singly linked list *in-place* — using only pointer manipulation (no extra array allowed). Print original and reversed list.

Input Format:

```
Line 1: N
Line 2: N space-separated integers
```

Output Format:

```
Original: ...
Reversed: ...
```

Sample Input / Output:

Input	Expected Output
5 1 2 3 4 5	Original: 1 2 3 4 5 Reversed: 5 4 3 2 1
1 42	Original: 42 Reversed: 42
4 10 20 30 40	Original: 10 20 30 40 Reversed: 40 30 20 10

 **Hint:** Use three pointers: *prev*, *curr*, *next*. Walk through the list once.

Problem 7: Merge Two Sorted Patient Queues [12 marks]

 **Medium**

 **Real-World Scenario:** A hospital has two separate queues of patients sorted by token number. Merge them into one sorted queue without using extra nodes — merge the existing nodes by adjusting pointers.

Task:

Given two sorted linked lists, merge them into one sorted linked list. Do not create new nodes; reuse existing ones by relinking. Print both input lists and the merged result.

Input Format:

```
Line 1: N (size of first list)
Line 2: N sorted integers
Line 3: M (size of second list)
Line 4: M sorted integers
```

Output Format:

```
List 1: ...
List 2: ...
Merged: ...
```

Sample Input / Output:

Input	Expected Output
3 1 3 5 4 2 4 6 8	List 1: 1 3 5 List 2: 2 4 6 8 Merged: 1 2 3 4 5 6 8
2 10 20 2 10 30	List 1: 10 20 List 2: 10 30 Merged: 10 10 20 30
0 3 5 10 15	List 1: (empty) List 2: 5 10 15 Merged: 5 10 15

Problem 8: Undo History — Find the Kth from Last Edit [12 marks]

● Medium

 **Real-World Scenario:** A text editor keeps a linked list of editing actions. The user wants to jump back K steps from the most recent edit (K -th from the end). Find that action efficiently.

Task:

Find the K -th node from the end of a linked list in a **single pass** (traverse the list only once, no second pass allowed). Print its value, or "Invalid" if K is out of range.

Input Format:

```
Line 1: N
Line 2: N space-separated integers
Line 3: K
```

Output Format:

```
Kth from end: X    OR    Invalid
```

Sample Input / Output:

Input	Expected Output
6 10 20 30 40 50 60 2	Kth from end: 50
5 1 2 3 4 5 1	Kth from end: 5
3 7 8 9 5	Invalid
4 100 200 300 400 4	Kth from end: 100

 **Hint:** Use two pointers separated by K nodes. When the leading pointer reaches the end, the trailing pointer is at the answer.

Problem 9: Rearrange a Job Schedule — Interleave Two Halves [16 marks]

 **Hard**

 **Real-World Scenario:** A job scheduler stores N tasks in a linked list. It must rearrange them so that task 1 is followed by task N , then task 2 followed by task $N-1$, and so on — all in-place. This pattern is used in balanced workload distribution across time slots.

Task:

Given a linked list $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$, rearrange it *in-place* to: $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$ **No extra array allowed.**

Strategy Hint (think through these steps):

- Find the middle of the list using slow/fast pointers.
- Reverse the second half in-place.
- Merge the two halves by alternating nodes.

Input Format:

```
Line 1: N  
Line 2: N space-separated integers
```


Output Format:

```
Before: ...  
After:  ...
```

Sample Input / Output:

Input	Expected Output
4 1 2 3 4	Before: 1 2 3 4 After: 1 4 2 3
5 1 2 3 4 5	Before: 1 2 3 4 5 After: 1 5 2 4 3
6 10 20 30 40 50 60	Before: 10 20 30 40 50 60 After: 10 60 20 50 30 40
2 7 8	Before: 7 8 After: 7 8
1 42	Before: 42 After: 42

 **Challenge:** This problem combines three techniques you have already practised in this assignment. Plan on paper before coding.

Marks Summary

#	Problem	Difficulty	Marks
1	Build and Print a Train Route	Easy	10
2	Count and Sum of Elements	Easy	10
3	Insert at Any Position	Easy	10
4	Delete by Value	Easy	10
5	Search and Report	Easy	10
6	Reverse the Delivery Route	Medium	10
7	Merge Two Sorted Patient Queues	Medium	12
8	Undo History — Kth from Last	Medium	12
9	Rearrange a Job Schedule	Hard	16
Total			100